Logging

Carsten Gips (HSBI)

TL;DR
Im Paket `java.util.logging` findet sich eine einfache Logging-API.
Über die Methode `getLogger()` der Klasse `Logger` (*Factory-Method-Pattern*) kann ein (neuer) Logger erzeugt werden, dabei wird über den String-Parameter eine Logger-Hierarchie aufgebaut analog zu den Java-Package-Strukturen. Der oberste Logger (der "Root-Logger") hat den leeren Namen.
Jeder Logger kann mit einem Log-Level (Klasse `Level`) eingestellt werden; Log-Meldungen unterhalb des eingestellten Levels werden verworfen.
Vom Logger nicht verworfene Log-Meldungen werden an den bzw. die Handler des Loggers und (per Default) an den Eltern-Logger weiter gereicht. Die Handler haben ebenfalls ein einstellbares Log-Level und verwerfen alle Nachrichten unterhalb der eingestellten Schwelle. Zur tatsächlichen Ausgabe gibt man einem Handler noch einen Formatter mit. Defaultmäßig hat nur der Root-Logger einen Handler.
Der Root-Logger (leerer String als Name) hat als Default-Level (wie auch sein Console-Handler) "`Info`" eingestellt.
Nachrichten, die durch Weiterleitung nach oben empfangen wurden, werden nicht am Log-Level des empfangenden Loggers gemessen, sondern akzeptiert und an die Handler des Loggers und (sofern nicht deaktiviert) an den Elternlogger weitergereicht.
Videos

- VL Logging
- Demo Logging (Überblick)
- Demo Log-Level
- Demo Logging: Handler und Formatter
- Demo Weiterleitung an den Elternlogger

---

1 Wie prüfen Sie die Werte von Variablen/Objekten?

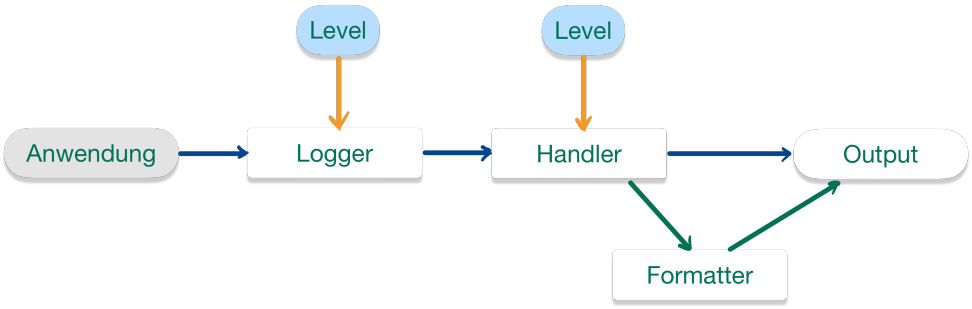
1. Debugging
   - Beeinflusst Code nicht
   - Kann schnell komplex und umständlich werden
   - Sitzung transient - nicht wiederholbar

2. "Poor-man's-debugging" (Ausgaben mit `System.out.println`)
   - Müssen irgendwann entfernt werden
   - Ausgabe nur auf einem Kanal (Konsole)
   - Keine Filterung nach Problemgrad - keine Unterscheidung zwischen Warnungen, einfachen Informationen, …

3. **Logging**
   - Verschiedene (Java-) Frameworks:
     `java.util.logging` (JDK), *log4j* (Apache), *SLF4J*, *Logback*, …

2 Java Logging API - Überblick

Paket `java.util.logging`



Eine Applikation kann verschiedene Logger instanziieren. Die Logger bauen per Namenskonvention hierarchisch aufeinander auf. Jeder Logger kann selbst mehrere Handler haben, die eine Log-Nachricht letztlich auf eine bestimmte Art und Weise an die Außenwelt weitergeben.

Log-Meldungen werden einem Level zugeordnet. Jeder Logger und Handler hat ein Mindest-Level eingestellt, d.h. Nachrichten mit einem kleineren Level werden verworfen.

Zusätzlich gibt es noch Filter, mit denen man Nachrichten (zusätzlich zum Log-Level) nach weiteren Kriterien filtern kann.

Konsole: logging.LoggingDemo

3 Erzeugen neuer Logger

```
import java.util.logging.Logger;
Logger l = Logger.getLogger(MyClass.class.getName());
```

- **Factory-Methode** der Klasse `java.util.logging.Logger`

  ```
  public static Logger getLogger(String name);
  ```

  => Methode liefert bereits **vorhandenen Logger** mit diesem Namen (sonst neuen Logger)

- **Best Practice**:
  Nutzung des voll-qualifizierten Klassennamen: `MyClass.class.getName()`

  - Leicht zu implementieren
  - Leicht zu erklären
  - Spiegelt modulares Design
  - Ausgaben enthalten automatisch Hinweis auf Herkunft (Lokalität) der Meldung
  - **Alternativen**: Funktionale Namen wie “XML”, “DB”, “Security”

4 Ausgabe von Logmeldungen

```
public void log(Level level, String msg);
```

- Diverse Convenience-Methoden (Auswahl):

```
public void warning(String msg)
public void info(String msg)
public void entering(String srcClass, String srcMethod)
public void exiting(String srcClass, String srcMethod)
```

- Beispiel

```
import java.util.logging.Logger;
Logger l = Logger.getLogger(MyClass.class.getName());
l.info("Hello World :-)");
```

5 Wichtigkeit von Logmeldungen: Stufen

- `java.util.logger.Level` definiert 7 Stufen:
  - `SEVERE`, `WARNING`, `INFO`, `CONFIG`, `FINE`, `FINER`, `FINEST`
    (von höchster zu niedrigster Prio)
  - Zusätzlich `ALL` und `OFF`

- Nutzung der Log-Level:
  - Logger hat Log-Level: Meldungen mit kleinerem Level werden verworfen
  - Prüfung mit `public boolean isLoggable(Level)`
  - Setzen mit `public void setLevel(Level)`

Konsole: logging.LoggingLevel

=> Warum wird im Beispiel nach `log.setLevel(Level.ALL);` trotzdem nur ab `INFO` geloggt? Wer erzeugt eigentlich die Ausgaben?!

6 Jemand muss die Arbeit machen ...

- Pro Logger **mehrere** Handler möglich
  - Logger übergibt nicht verworfene Nachrichten an Handler

  - Handler haben selbst ein Log-Level (analog zum Logger)
  - Handler verarbeiten die Nachrichten, wenn Level ausreichend

- Standard-Handler: `StreamHandler`, `ConsoleHandler`, `FileHandler`

- Handler nutzen zur Formatierung der Ausgabe einen `Formatter`
- Standard-Formatter: `SimpleFormatter` und `XMLFormatter`

Konsole: logging.LoggingHandler

=> Warum wird im Beispiel nach dem Auskommentieren von `log.setUseParentHandlers(false);` immer noch eine zusätzliche Ausgabe angezeigt (ab `INFO` aufwärts)?!

7 Ich ... bin ... Dein ... Vater ...

- Logger bilden **Hierarchie** über Namen
  - Trenner für Namenshierarchie: "`.`" (analog zu Packages) => mit jedem "`.`" wird eine weitere Ebene der Hierarchie aufgemacht ...
  - Jeder Logger kennt seinen Eltern-Logger: `Logger#getParent()`
  - Basis-Logger: leerer Name (`""`)
    * Voreingestelltes Level des Basis-Loggers: `Level.INFO` (!)

- Weiterleiten von Nachrichten
  - Nicht verworfene Log-Aufrufe werden an Eltern-Logger weitergeleitet (Default)
    * Abschalten mit `Logger#setUseParentHandlers(false);`
  - Diese leiten an ihre Handler sowie an ihren Eltern-Logger weiter (unabhängig von Log-Level!)

Konsole: logging.LoggingParent; Tafel: Skizze Logger-Baum

8 Wrap-Up

- Java Logging API im Paket `java.util.logging`

- Neuer Logger über **Factory-Methode** der Klasse `Logger`
  - Einstellbares Log-Level (Klasse `Level`)
  - Handler kümmern sich um die Ausgabe, nutzen dazu Formatter
  - Mehrere Handler je Logger registrierbar
  - Log-Level auch für Handler einstellbar (!)
  - Logger (und Handler) "interessieren" sich nur für Meldungen ab bestimmter Wichtigkeit
  - Logger reichen nicht verworfene Meldungen defaultmäßig an Eltern-Logger weiter (rekursiv)

9 Zum Nachlesen

- Oracle Corporation (2022, Kap. 8)

Lernziele

- k3: Nutzung der Java Logging API im Paket java.util.logging
- k3: Erstellung eigener Handler und Formatter

Quizzes

- Quiz Logging (ILIAS)

Challenges

1. Schreiben Sie einen Formatter, welcher die Meldungen in folgendem Format auf der *Konsole* ausgibt. Bauen Sie diesen Formatter in alle Logger ein.

```
------------
Logger: record.getLoggerName()
Level: record.getLevel()
Class: record.getSourceClassName()
Method: record.getSourceMethodName()
Message: record.getMessage()
------------
```

2. Schreiben Sie einen weiteren Formatter, welcher die Daten als Komma-separierte Werte (CSV-Format) mit der folgenden Reihenfolge in eine *Datei* ausgibt (durch Anfügen einer neuen Zeile an bereits bestehenden Inhalt). Bauen Sie diesen Formatter in den Logger für den Ringpuffer ein.

```
record.getLoggerName(),record.getLevel(),record.getSourceMethodName()
    ,record.getSourceClassName(),record.getMessage()
```

3. Ersetzen Sie in einem Beispielprogramm sämtliche Konsolenausgaben (`System.out.println` und `System.err.println`) in der Vorgabe durch geeignete Logger-Aufrufe mit passendem Log-Level.

   Alle Warnungen und Fehler sollen zusätzlich in eine `.csv`-Datei geschrieben werden. Auf der Konsole sollen alle Log-Meldungen ausgegeben werden.

Quellen

Oracle Corporation. 2022. „Java Core Libraries Developer Guide“. 2022. https://docs.oracle.com/en/java/javase/17/core/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 9d7ab85 (Remove Hugo: remove pandoc escaping on video links, 2025-04-30)